1 Image Classification Pipeline

1.1 Overview

At a high level, the pipeline:

1. **Loads** a pre-trained ResNet50 classifier.
2. **Reads and transforms** each image in a directory.
3. **Runs** the model to get class probabilities.
4. **Maps** the predicted class to one or more cuisine types.
5. **Counts** how often each cuisine appears.
6. **Outputs** the top 3 cuisine type guesses.

This approach allows to classify many images and get an aggregate guess of the predominant cuisine types present.

1.2 Preparing Input Images

1. **List** all files in the target directory.
2. **Filter** by image extensions (`.png`, `.jpg`, `.jpeg`).
3. **Open** each image with PIL and convert to RGB.
4. **Apply** the `transform` to get a tensor.

1.3 Running Inference

1. **Forward pass** through the model to get raw logits.
2. **Apply softmax** to convert logits to probabilities.
3. **Select** the top-1 class and its probability.

```
with torch.no_grad():
    logits = model(image_tensor)              # [1, out_features]
    probs = torch.nn.functional.softmax(logits, dim=1)
    top_prob, top_idx = probs.max(dim=1)
    class_index = top_idx.item()
    confidence = top_prob.item()
```

- **logits**: Raw model outputs.
- **probs**: Normalized confidences for each class.
- **class_index**: Index of the most likely class.
- **confidence**: Probability of that class (0–1).

1.4 Mapping Predictions to Cuisine Types

We maintain a JSON file (**food_cuisine_mapping.json**) that maps each class name to a list of cuisine types. For example:

```
{
  "lasagna": ["italian", "argentine", "brazilian"],
  "nachos": ["mexican", "canadian"],
  // ...
}
```

1.5 Aggregating and Displaying Results

As we classify each image, we tally the counts for each cuisine type:

```
cuisine_counts = {}
for cuisine in cuisines:
    cuisine_counts[cuisine] = cuisine_counts.get(cuisine, 0) + 1
```

After processing all images, we sort the cuisine types by count and display the top 3:

Rank	Cuisine	Count
1	Italian	10
2	French	7
3	Japanese	5

This gives you a concise summary of which cuisines appeared most often in your dataset.

1.6 Idea behind the Pipeline

We use a voting system to aggregate predictions across many images. This helps us: - **Reduce noise**: Individual images may be misclassified, but aggregating many predictions smooths out errors. - **Capture diversity**: A single restaurant may serve multiple cuisines, so we want to see the overall trends. - **Provide confidence**: The top cuisines with counts give a clearer picture than just a single guess.

Also does a *Top-3* help us with uncertain predictions. If the top guess is not very confident, we can still see other likely cuisines.